Booking-Agent ("Tazkara") — Capstone Project Report

Team (Group 12): Nawaf Almufarej (lead) · Dana · Hessa · Refal
Program: Agentic AI Bootcamp — Capstone
Repository: chat-first AI ticket-booking concierge · Python 3.11 · FastAPI · Next.js 14

---

1. What we built

Booking-Agent ("Tazkara") is a chat-first AI concierge that books live-event tickets in Saudi Arabia inside a single conversation. Instead of a multi-page checkout, the user just talks to the agent, which carries them end to end:

> ***search → identify member (by email) → quote (member discount + 15% VAT) → seat map → atomic 10-minute hold → confirm (human-in-the-loop) → pay → signed QR ticket.***

The system ships with a deterministic Saudi demo catalog (Coldplay Riyadh/Jeddah, the Riyadh Derby, LEAP, MDLBEAST Soundstorm, a weekend comedy night), a FastAPI backend, two frontends (a Next.js storefront and a no-Node static site), a Typer CLI, and a test suite of **159 passing tests**.

2. Who it is for

- **Saudi event-goers** who want to go from "I want a ticket" to "ticket in my inbox" in under two minutes — in Arabic, English, or a mix — with their member discount applied automatically.
- **Organisers / a platform like WeBook**, which has a dense catalog but converts worse than it should because the experience is generic. The agent's differentiators (preference memory + real-time sentiment adaptation) target exactly that conversion gap.

3. How it works

Every chat turn enters one pipeline (`agent/__init__.py:handle`) that dispatches on a configured mode:

- `fsm` **(default)** — a deterministic state machine over typed Python tools. It extracts slots, advances as far as the message allows, and asks for the first missing one (event → email → category → quantity → seats → confirm). This is the MVP and what every test exercises.
- `tool_agent` — a genuinely agentic LLM **Reason–Act–Observe loop**: the model chooses which tool to call next, with a step cap to prevent runaway loops. It falls back to the FSM automatically when no LLM key is present.

In **both** modes the dangerous action — creating the booking and issuing the payment — is **kept out of the model's reach** and executed in code **only** after an explicit user "confirm". This human-in-the-loop gate is a non-negotiable design principle, not a prompt instruction.

Three engineering invariants underpin the flow:

Invariant	How it is enforced
Exact money	All amounts are integer *halalas* (1 SAR = 100), discounts integer *basis points* (1500 = 15%). No float ever enters the arithmetic; quotes are tested to the halala.
Atomic seat holds	A hold is all-or-nothing with a 10-minute TTL — if any requested seat is taken it raises and holds nothing, so two bookings can never hold the same seat. A sweeper releases expired holds.
Tamper-evident tickets	The QR encodes an HMAC-signed token (booking id + seats + nonce + signature), not a bare id — a screenshot can't be forged or replayed.

Persistence is SQLAlchemy 2 over 13 tables (events, venues, seats, members, bookings, holds, payments, tickets, audit/interaction logs, …), SQLite by default and Postgres-compatible. Seat maps are rendered to PNG on demand with Pillow (green = available, amber = held, grey = sold).

4. AI components used

Component	Role	Notes
LLM slot extraction (`agent/llm.py`)	Turns free text into structured intent + slots	Provider-agnostic: Anthropic / OpenAI / Nano-GPT
Tool-calling agent (`agent/tool_agent.py`)	Reason–Act–Observe loop; model picks tools	Loop-prevention cap; HITL gate outside model control
Rule-based fallback (`agent/extract.py`)	Deterministic NLU when no key / network	Lets the whole product (and test suite) run fully offline
Reply composition & Q&A (`compose.py` , `answer.py`)	Natural rephrasing and chit-chat / FAQ answers	Isolated, timeout-bounded LLM calls
Sentiment & personalisation (`sentiment.py` , `interests.py`)	Per-turn engaged/neutral/frustrated; preference weighting	The "concierge, not chatbot" differentiators
RAG + LLM-as-judge (`rag.py` , `judge.py`)	Venue-FAQ retrieval; grading the agent on an eval set	Eval harness scores live-model runs

The design deliberately isolates every LLM-touched concern into its own module and keeps a deterministic fallback, so a slow or missing provider degrades gracefully instead of breaking the booking flow.

5. What worked well

- **The full happy-path loop works end to end** — search through signed QR ticket — in both the Next.js and static UIs and from the CLI.

- **The human-in-the-loop payment gate holds** in every mode; no payment link is ever issued without explicit confirmation.
- **Money and seat-hold correctness** are rock-solid: integer-halala math and atomic holds are covered by tests (including concurrency and TTL/expiry via `freezegun`). **All 159 tests pass.**
- **Graceful degradation**: with no LLM key, the agent transparently uses the rule-based path — the demo and tests run offline with zero network.
- **Spec-driven build**: every feature was sliced as spec → plan → tasks under `specs/`, which kept scope honest.

6. What did not work / is still incomplete

- **Live payment**: the demo uses an offline `fake` gateway. The real **Moyasar** sandbox webhook (signature verification + idempotency) is specified and partly wired but not fully integrated.
- **PDF ticket + email delivery**: the QR token signing and PNG rendering work, but the ReportLab PDF layout and SMTP send are not finished.
- **Personalisation + sentiment (F006)** and **RAG venue-FAQ (F007)** are specified and partially scaffolded, but not part of the default flow yet.
- **Sessions are in-memory**, so conversational state does not survive a server restart.

7. What we would improve next

1. **Finish the Moyasar webhook** with verified-signature + idempotency so a paid booking is confirmed exactly once, then complete the **PDF ticket + email**.
2. **Ship F006** end to end — persistent per-user preference profiles and sentiment-driven strategy switching — and measure the conversion lift.
3. **Persist sessions** (Redis or DB) and add reconnect/resume.
4. **Bilingual evaluation set** for Arabic/English code-switching, plus broader LLM-as-judge coverage of tool-selection correctness.
5. **Move toward the multi-agent (LangGraph) design** reserved as the stretch goal.

Setup and run instructions are in `02_code/README.md`; the original plan is in `01_proposal/`; a recorded walkthrough is in `04_presentation/`.